

PROGRAMAÇÃO ORIENTADA A OBJETOS - CHECKPOINT 1

Nome: _____

Nota: _____

RESPOSTAS

QUESTÃO 1	a	b	c	d	e
QUESTÃO 2	a	b	c	d	e
QUESTÕES 3-7	3	4	5	6	7
QUESTÕES 8-12	8	9	10	11	12

CENÁRIO: ELEVADOR

Os arquivos `Elevador.java` e `Main.java` estão no mesmo pacote. Considere exatamente o código abaixo.

Elevador.java	Main.java
<pre> 1 public class Elevador { 2 int andarAtual = 0; 3 boolean portaAberta = false; 4 5 void subir() { 6 andarAtual = andarAtual + 1; 7 } 8 void abrirPorta() { 9 portaAberta = true; 10 } 11 void fecharPorta() { 12 portaAberta = false; 13 } 14 int getAndarAtual() { 15 return andarAtual; 16 } 17 boolean getPortaAberta() { 18 return portaAberta; 19 } 20 }</pre>	<pre> 1 public class Main { 2 public static void main(String[] args) { 3 Elevador principal = new Elevador(); 4 Elevador painel = principal; 5 Elevador servico = new Elevador(); 6 7 principal.subir(); 8 painel.abrirPorta(); 9 10 System.out.println(principal.getAndarAtual()); 11 System.out.println(principal.getPortaAberta()); 12 13 painel.fecharPorta(); 14 painel.subir(); 15 16 System.out.println(painel.getAndarAtual()); 17 System.out.println(painel.getPortaAberta()); 18 19 servico.subir(); 20 servico.abrirPorta(); 21 22 System.out.println(servico.getAndarAtual()); 23 } 24 }</pre>

QUESTÕES

Questão 1 (25 pontos)

Faça o teste de mesa. Para cada linha indicada, escreva exatamente o valor impresso.

- | | |
|-------------|-------------|
| a) Linha 10 | d) Linha 17 |
| b) Linha 11 | e) Linha 22 |
| c) Linha 16 | |

Questão 2 (25 pontos)

A classe agora protege os campos e controla a subida. Considere somente o código abaixo e marque cada afirmação como V ou F.

Os getters, `abrirPorta()` e `fecharPorta()` continuam iguais aos do cenário.

Trechos alterados em Elevador.java
<pre> private int andarAtual; private boolean portaAberta; int subir() { if (portaAberta == false) { if (andarAtual < 3) { andarAtual = andarAtual + 1; } } return andarAtual; }</pre>

- Em `Main`, a instrução `principal.andarAtual = 2;` compila.
- Porta fechada, andar 2: `int resultado = principal.subir();` deixa resultado e o elevador no andar 3.
- Porta aberta, andar 1: `subir()` retorna 1 e preserva o andar 1.
- Porta fechada, andar 3: duas chamadas de `subir()` retornam 3 e preservam o andar 3.
- `Main` pode consultar o campo privado pelo método `getAndarAtual()`.

Questão 3 (5 pontos)

O sistema também registra chamados de manutenção, com descrição, prioridade, situação e a ação de encerrar. Qual alternativa mantém juntos esses dados e essa ação?

- A. Manter descrição, prioridade e situação em variáveis separadas de Main e encerrar o chamado ali.
- B. Criar uma classe apenas para a descrição e manter prioridade, situação e encerramento em Main.
- C. Criar ChamadoManutencao com descrição, prioridade, situação e um método para encerrar o chamado.
- D. Criar uma classe para cada dado do chamado e realizar o encerramento em Main.
- E. Manter os três dados em Main e passá-los a um método externo sempre que o chamado mudar.

Questão 4 (5 pontos)

Na instrução Elevador principal = new Elevador();, qual alternativa identifica corretamente a classe, a variável e o objeto criado?

- A. Elevador é a variável; principal é a classe; new copia um objeto.
- B. Elevador é a classe; principal guarda uma referência; new Elevador() cria o objeto.
- C. principal é o objeto; Elevador guarda a referência usada por new.
- D. new Elevador() é a variável; principal declara outra classe.
- E. Os três nomes representam o mesmo papel e identificam diretamente o objeto.

Questão 5 (5 pontos)

Quando abrirPorta() é executado, o valor de portaAberta muda. Qual alternativa classifica corretamente o campo e o método?

- A. portaAberta é estado; abrirPorta() é comportamento.
- B. portaAberta é comportamento; abrirPorta() é estado.
- C. Ambos são estados porque pertencem ao mesmo objeto.
- D. Ambos são comportamentos porque podem ser acessados com ponto.
- E. portaAberta é variável local; abrirPorta() é método de Main.

Questão 6 (5 pontos)

Queremos um método que receba um andar de destino inteiro e não retorne valor. Qual assinatura atende ao requisito?

- A. int irPara()
- B. void irPara()
- C. int irPara(int destino)
- D. void irPara(int destino)
- E. void irPara(float portaAberta)

Questão 7 (5 pontos)

Observe o código. Ele compila? Se não, qual linha impede a compilação?

```
Trecho em Elevador.java
1 int andarAtual;
2 void mostrar() {
3     int anterior;
4     System.out.println(andarAtual);
5     System.out.println(anterior);
6 }
```

- A. Compila e as linhas 4 e 5 imprimem 0.
- B. Não compila: a linha 5 tenta ler a variável local anterior sem inicializá-la.
- C. Não compila: a linha 4 tenta ler o campo andarAtual sem inicializá-lo.
- D. Compila, mas a execução falha na linha 5 ao acessar anterior.
- E. Não compila: apenas declarar a variável local na linha 3 já é inválido.

Questão 8 (5 pontos)

Elevador terá o campo int quantidadePessoas; a lotação máxima é 8. Onde deve ficar a decisão de aceitar ou rejeitar um grupo?

- A. Main consulta a lotação e decide se inclui o grupo.
- B. Um auxiliar de Main calcula e altera a lotação.
- C. Main verifica o grupo e outro método externo verifica o limite.
- D. Um setter recebe a nova lotação calculada fora do objeto.
- E. Elevador.embarcar(quantidade) verifica sua lotação e o limite.

Questão 9 (5 pontos)

Depois de executar a última linha abaixo, quais são os valores de diferenca e andarAtual?

```
Trecho em Elevador.java e Main.java
1 // Em Elevador.java
2 int calcularDiferenca(int destino) {
3     return destino - andarAtual;
4 }
5 // Em Main.java
6 Elevador teste = new Elevador();
7 teste.subir();
8 int diferenca = teste.calcularDiferenca(3);
```

- A. diferenca recebe 3 e andarAtual passa a ser 3.
- B. diferenca recebe 2 e andarAtual passa a ser 3.
- C. Não compila: um método com retorno int não pode receber parâmetro.
- D. diferenca recebe 2 e andarAtual continua em 1.
- E. O método não retorna valor porque não foi declarado com void.

Questão 10 (5 pontos)

Após executar o trecho, o que é correto sobre os estados de a e b e sobre a == b?

```
Trecho em Main.java
1 Elevador a = new Elevador();
2 Elevador b = new Elevador();
3 a.subir();
```

- A. Antes da chamada, a == b é true; depois, ambos ficam no andar 1.
- B. Antes da chamada, os estados diferem; depois, somente b fica no andar 1.
- C. Antes da chamada, os estados são iguais e a == b é false; depois, somente a fica no andar 1.
- D. Antes da chamada, a == b é true; depois, somente a fica no andar 1.
- E. Antes da chamada, os estados são iguais; por isso, alterar a também altera b.

Questão 11 (5 pontos)

Uma classe de outro pacote tenta chamar getAndarAtual(), que não tem modificador de acesso. O que acontece?

- A. Compila porque a classe Elevador é pública.
- B. Não compila: o acesso de pacote permite uso apenas no mesmo pacote.
- C. Só compila se o campo andarAtual também for público.
- D. O método se torna privado automaticamente por ser chamado de outro pacote.
- E. Compila porque a ausência de modificador equivale a public.

Questão 12 (5 pontos)

Os andares válidos vão de 0 a 3. Após acrescentar o método abaixo, Main chama principal.definirAndar(8). O que acontece?

```
Trecho acrescentado em Elevador.java
1 public void definirAndar(int novoAndar) {
2     andarAtual = novoAndar;
3 }
```

- A. Não compila: a própria classe não pode alterar um campo privado.
- B. Não compila: o valor 8 está fora do intervalo permitido.
- C. Compila, e andarAtual passa automaticamente para 3.
- D. Compila, e andarAtual passa para 8 porque não há validação.
- E. Compila, mas andarAtual continua em 0 porque o método é void.